

NclaveOS Agentic Operating System

An Early-Access Open Source Project Made by Exxeta

| *KI-Agenten für Teams — governed, auditierbar, self-hosted*



OPA Policy Gate



Multi-User RBAC



Secret Isolation



Approval Gate



Audit Trail



Scheduled Tasks

Das Problem

Jeder baut sich eigenen Agenten Stack

Jeder konfiguriert lokal — keine gemeinsamen Skills, keine einheitlichen Policies.

Kein Audit

Was hat der Agent auf Prod gemacht? Weiß keiner.

Secrets im Prompt

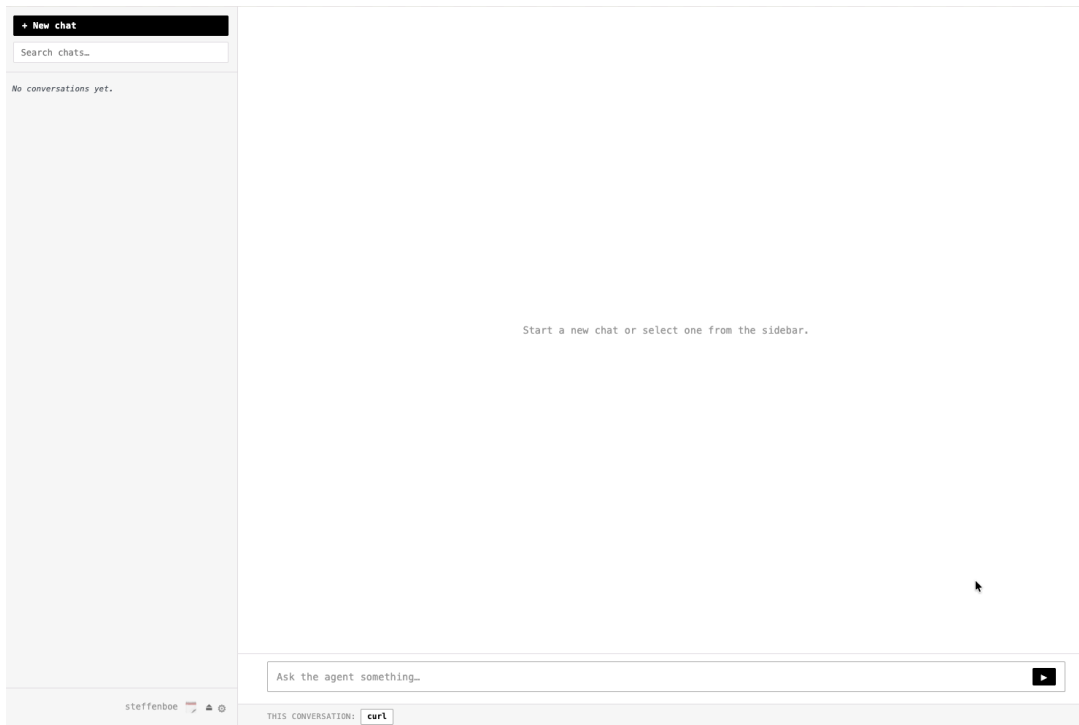
API-Keys landen im LLM-Kontext, in Logs, in der Bash-History.

Kein Gate

Der Agent kann Prod-Ressourcen löschen — nichts hält ihn auf.

Introducing: NclaveOS Agentic Operating System

"From personal automation to governed agent infrastructure."



- **zentral gehostetes** Agenten-Management
- Agenten **planen und führen Commands aus**
- Skills und Systemzugriffe der Agenten werden **zentral verwaltet**
- **deterministische Guardrails** zentral konfigurierbar
- opt. **Approval-Gate** vor Commands
- **Audit Trails**
- Cron-Jobs
- ... *und mehr geplant*

Lokaler Agent vs. NclaveOS

Fähigkeit	Claude Desktop / MCP	nclaveOS
Multi-User mit Rollen	×	✓
Policy-gated Execution	×	✓ OPA Rego
Secret-Isolation	×	✓
Human Approval Gate	✓	✓
Scheduled Tasks	×	✓ Cron
Zentraler Audit Trail	×	✓
Gemeinsame Skill-Bibliothek	×	✓
Self-hosted & OSS	teilweise	✓ immer

Wie es funktioniert

 Prompt	→	 Planner (LLM)	→	 OPA Policy	 →	 Executor	→	 Audit Trail
		↑ result		 deny				

Planner → OPA → Executor → result zurück an Planner — Loop bis done oder failed

Skills

```
name: kubectl-readonly
description: |
  Read-only k8s CLI.
  Only get, describe, logs, top.
policy: |
  allowed := {"get","describe","logs","top"}
  allow {
    input.argv[0] == "kubectl"
    allowed[input.argv[1]]
  }
env:
  - KUBECONFIG_TOKEN
```

Admins definieren:

- welche Tools verfügbar sind
- welche Commands erlaubt sind
- welche Secrets injiziert werden

Verteilt via Git-Repository →

ein Sync-Klick, alle Teams haben dieselben Skills.

OPA als Ausführungsgate

Command	OPA	Ergebnis
<code>kubectl delete pod x</code>	<code>default allow = false</code>	 blocked
<code>kubectl get pods -n prod</code>	<code>allowed := {"get"}</code>	 executed
<code>rm -rf /</code>	kein Skill matched	 blocked

Globale Policy + optionale **per-Skill-Policy** — in-process, kein Netzwerk-Round-Trip.

Secret-Isolation

	sieht
 LLM	<code>\${GITHUB_TOKEN}</code> — Placeholder
 Audit Trail	<code>\${GITHUB_TOKEN}</code> — Placeholder
 Subprocess	<code>GITHUB_TOKEN=ghp_xxxx</code> — echter Wert
<code>secrets.json</code>	<code>chmod 600</code> , gitignored — nur der Server liest es

Secrets verlassen **nie** den Subprocess — nicht im LLM-Kontext, nicht in Logs, nicht in der Run-History.

Browser-UI

Nutzer

-  Chat-Interface
-  Approve / Deny per Command
-  Run-History durchsuchen
-  Skills pro Chat aktivieren
-  Scheduled Tasks (Cron)

Admins

-  User-Verwaltung + Rollen
-  LLM-Endpoint & Modell
-  Skills (inkl. Guardrails) erstellen, testen und ausrollen
-  Approval global ein/ausschalten

Deployment

```
docker compose up -d
```

	Option A	Option B
LLM-Backend	OpenAI / Azure OpenAI	Ollama (lokal)
Persistenz	JSON Files (default)	MongoDB via MONGODB_URI
Port	:8081	:8081

Roadmap

Feature	Status
Per-Chat Model Selection	 In Planung
Webhook-Trigger	 In Planung
Kubernetes Helm Chart	 In Planung
Skill Marketplace	 Idee

Mitmachen

Nutzer

1. `docker compose up -d`
2. Skill laden
3. Run starten

Contributor

1. Fork + Feature Branch
2. `pytest` / `npm run test`
3. Pull Request

MIT License · © 2026 Exxeta AG · github.com/exxeta/nclaveos